Technische Universität Berlin

Distributed Systems
Chapter 2: Cooperation and Coordination

FG Verteilte Systeme und Betriebssysteme | Distributed Systems

Agenda

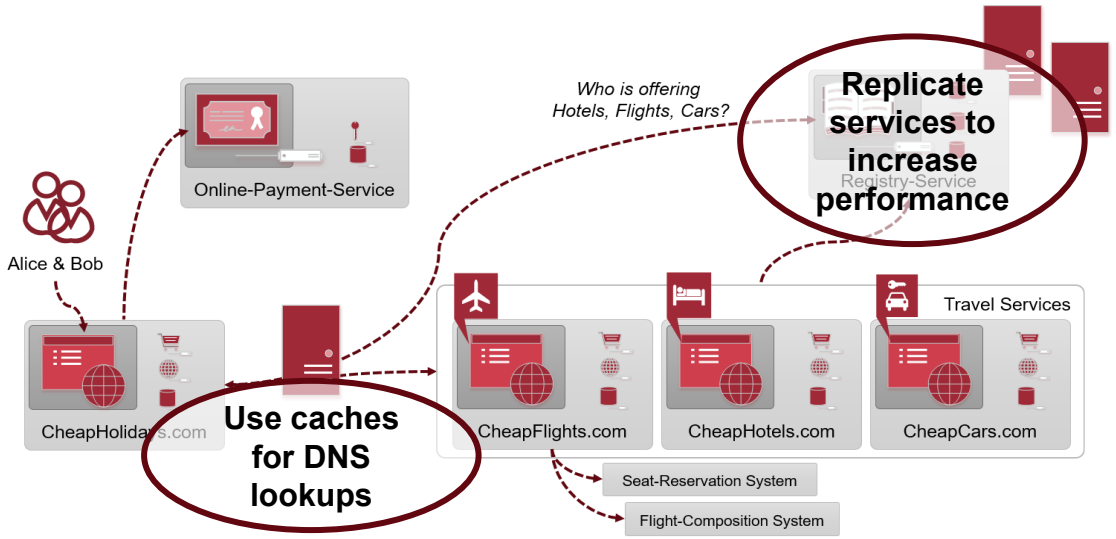
2.3 Replication

Replication - Motivation

Replication is one of the core DS capabilities to improve services

Performance enhancement

- Use data-caches close to clients (e.g. proxy servers or browser cache)
- Replicate data between servers and distribute request using load-balancing
- Replication of changing data introduces overhead to ensure that clients receive up-to-date results

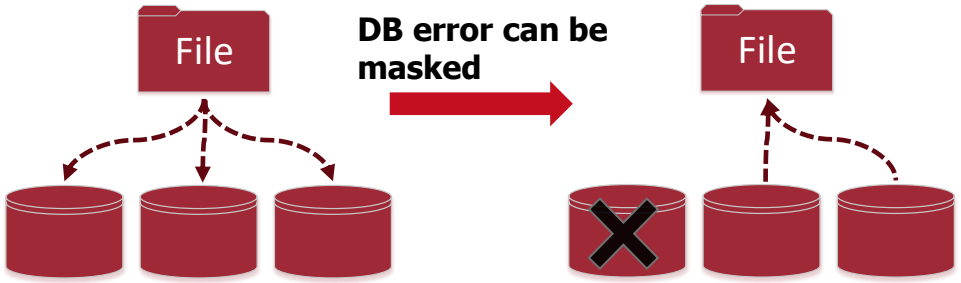


Replication - Increased (High) Availability

The proportion of time for which a service is available with reasonable response time should be close to 100%

Replication is a technique for automatically maintaining data availability in the presence of server failures (e.g. RAIDs)

Local replicas can be used to continue working in case of disconnection (Attention: consistency!)



Replication - Fault Tolerance

High availability (HA) data is not necessarily correct data → conflicting updates in case of disconnection or network partition

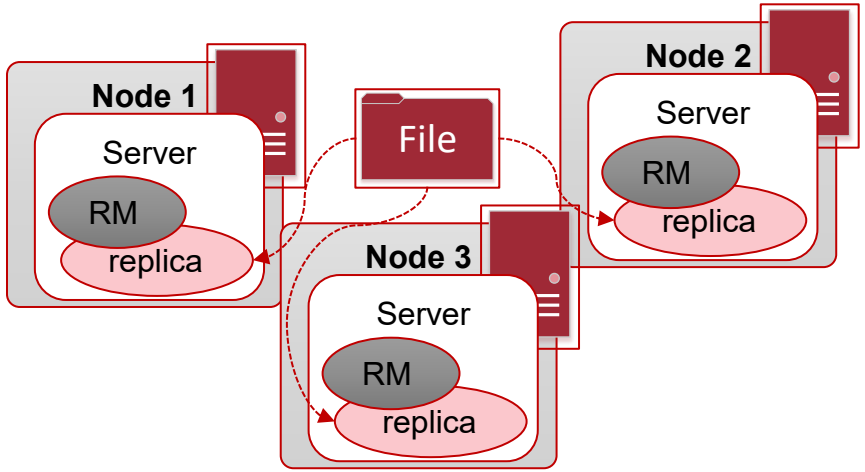
Fault tolerance concerns

- Correctness of data (i.e. freshness of data delivered to the client) → Consistency and Byzantine failures
- Timeliness of service responses (similar to HA)

Replication – System Model

Data consists of a collection of items/objects (e.g. files)

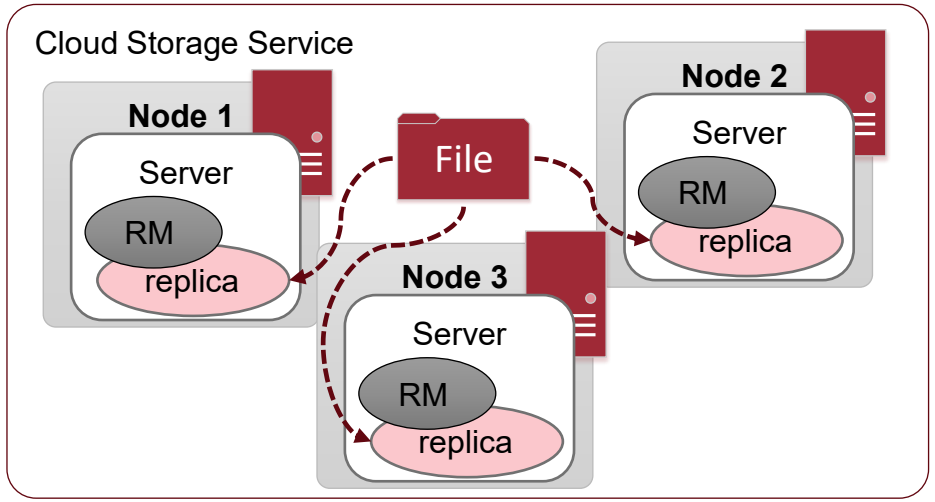
- Each logical object is implemented by a set of physical copies called replicas
- Each replica is held by a replica manager (RM) that applies recoverable operations to it
- No inconsistent results in case an operation fails (e.g. corrupted files)
- Replicas are not necessarily identical (at least not at any particular point in time)



Replication – System Model

Replica managers together constitute/provide a service to the clients

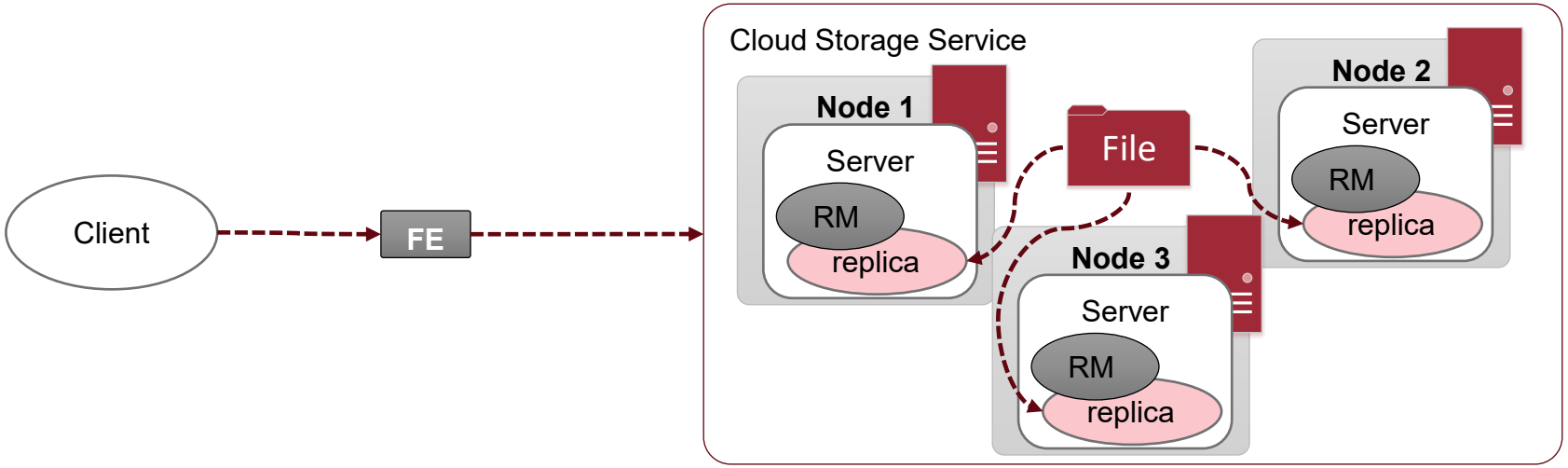
- The service offers access to logical objects
- Clients request a series of operations on the objects
- Read-only requests involve no updates, update requests involve updates and reads



Replication – System Model

Client requests are handled by components called front ends

- Front Ends (FE) are enablers for replication transparency
- FE receive client requests and forward these to one or more RMs (based on the application requirements)

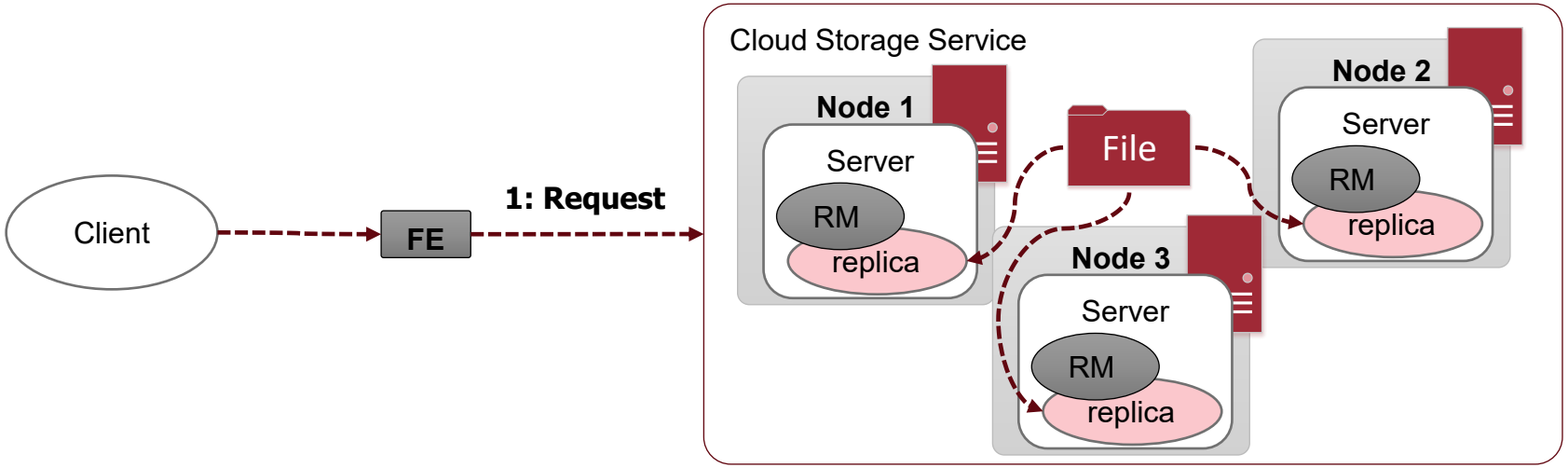


Replication – System Model

Five actions are involved in executing a single client request

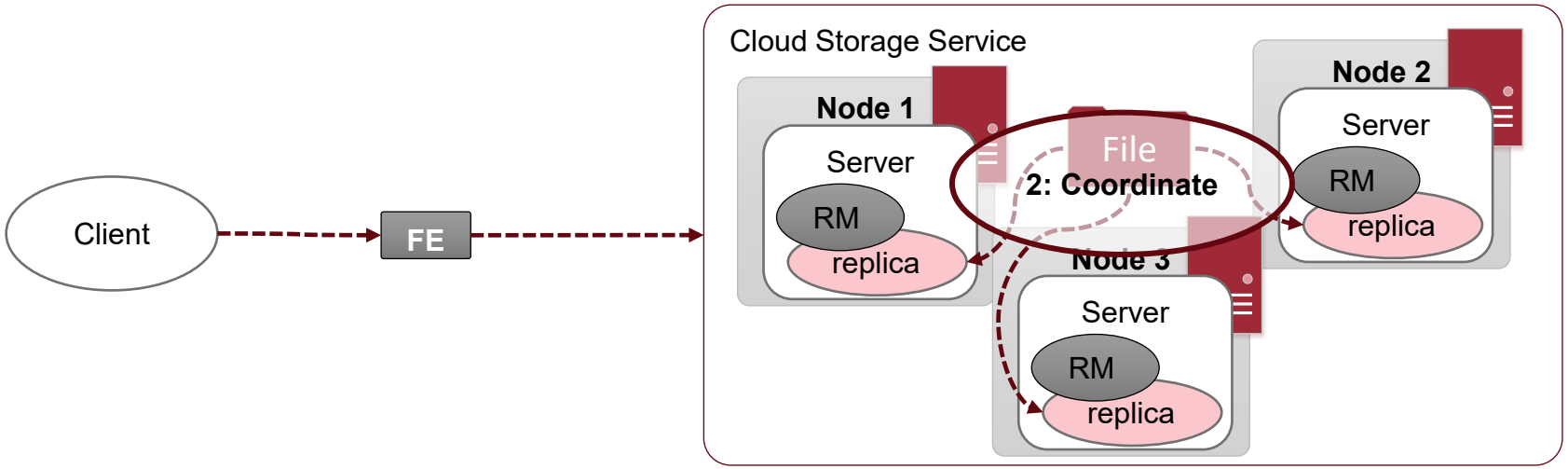
(1) Request: FE issues the request to one or more RMs

- Either FE communicates with one RM, which then communicates with others
- Or FE multicasts request to RMs



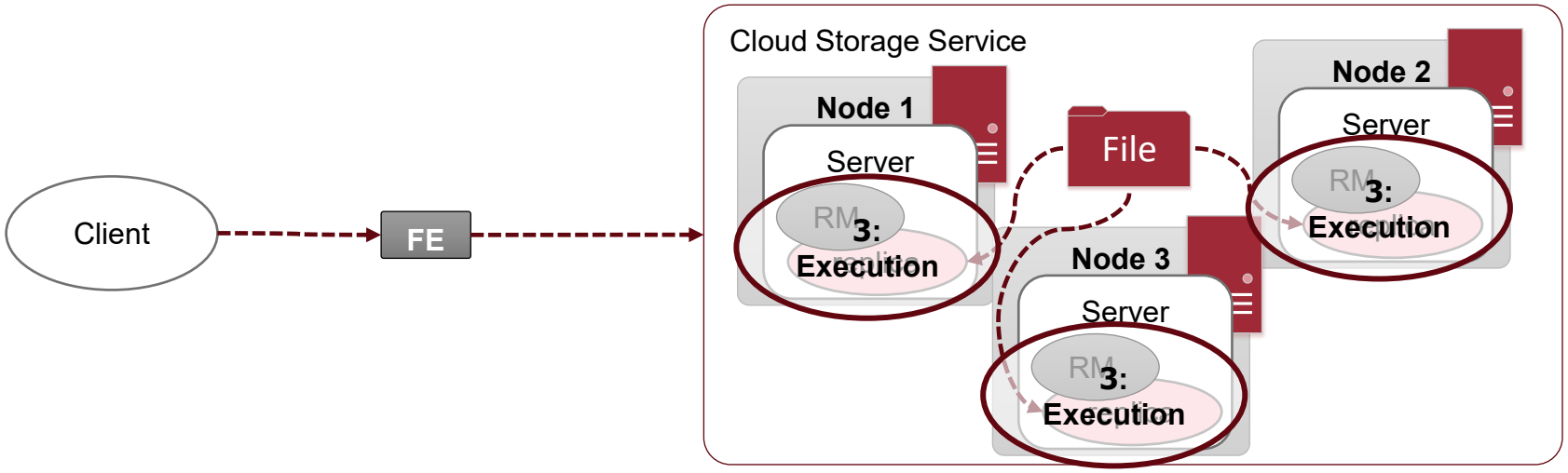
Replication – System Model

(2) Coordination: RMs coordinate before executing request to ensure for consistency, e.g. all RMs lock the replicas for an update request



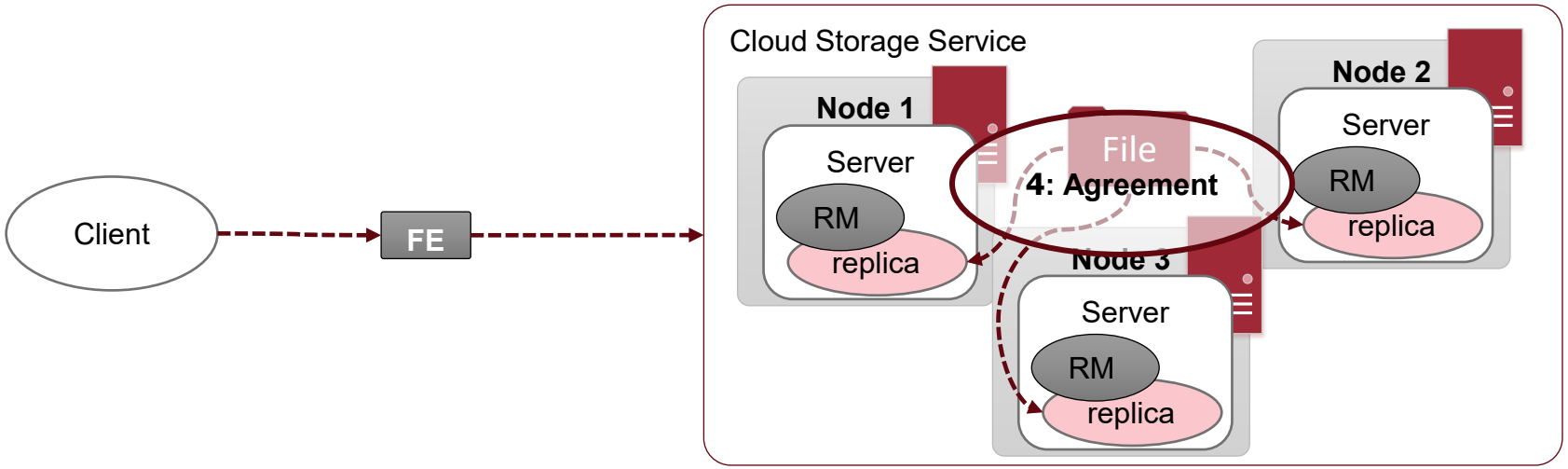
Replication – System Model

(3) Execution: RMs execute request (provisional – i.e. effects of request can be undone later)



Replication – System Model

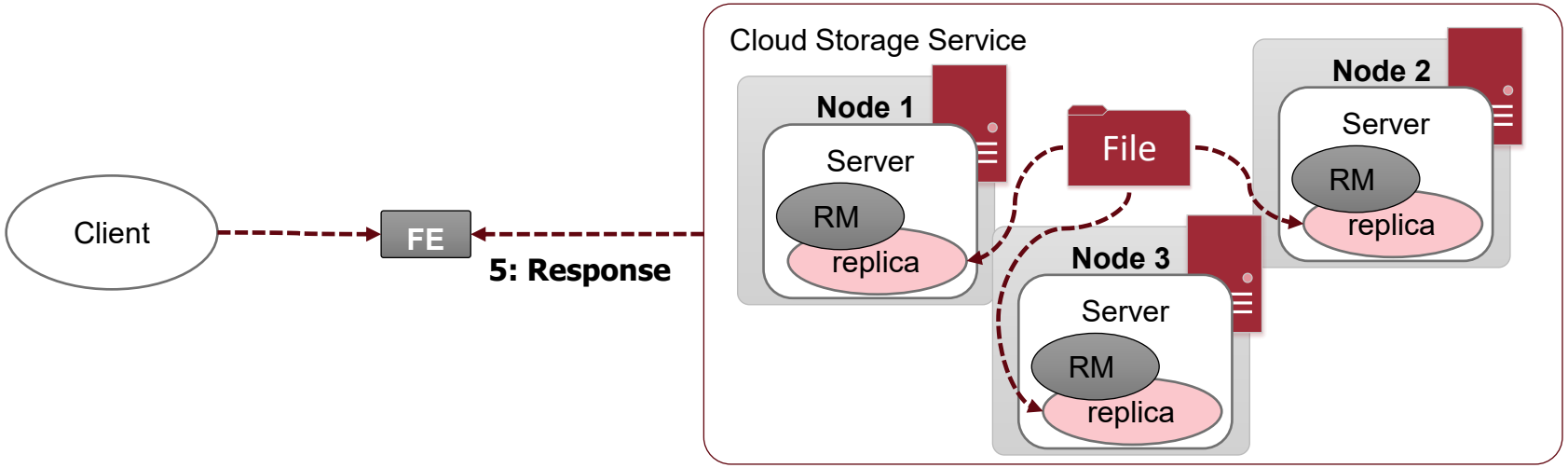
(4) Agreement: RMs reach consensus on the effect of the request, e.g. can the changes be made permanent



Replication – System Model

(5) Response: one or more RMs respond to the FE

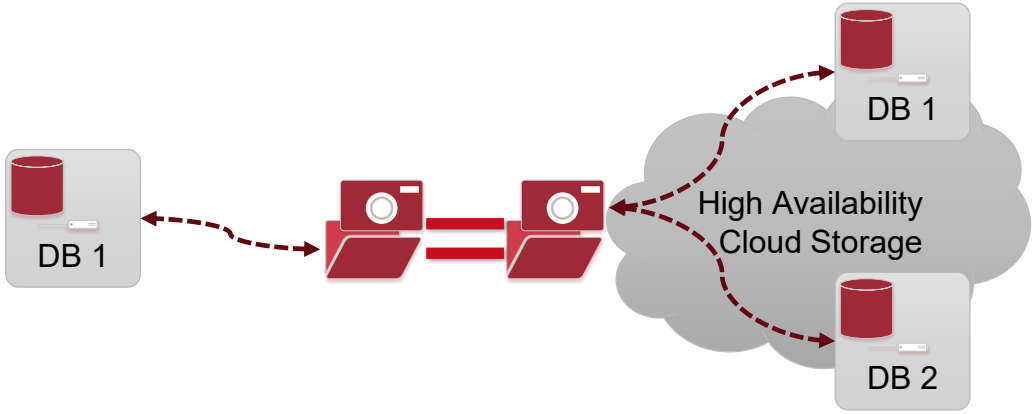
- In some applications one RM sends the results
- In others, the FE selects or aggregates the appropriate result (e.g. first result in case of time critical HA service)



Replication – Consistency

A fault tolerant service based on replication is correct, if

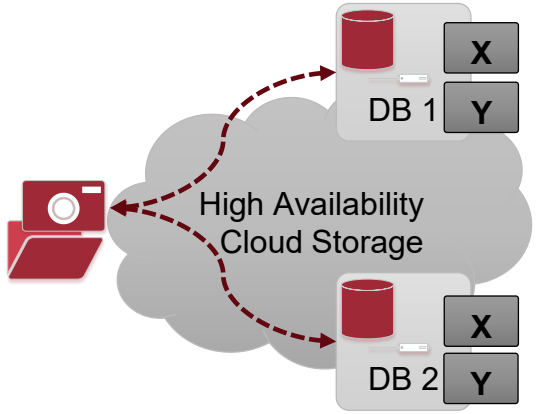
- It keeps responding despite failures
- Clients cannot observe a difference between the service they obtain from an implementation with replicated data and one provided by a single correct RM
- Consistency must be maintained among replicas



Replication – Consistency

An example of inconsistency

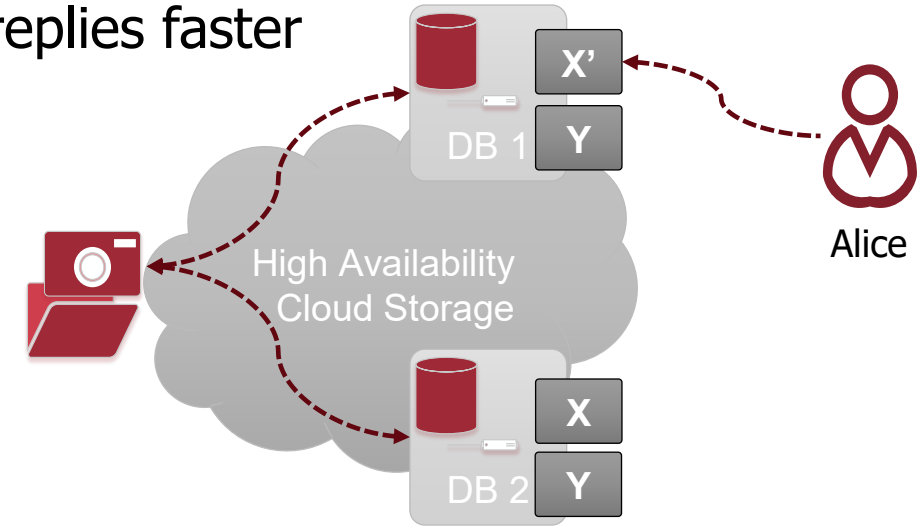
- Alice & Bob store two pictures X and Y
- Both DB1 and DB2 maintain a replica of X and Y
- Alice works at home and modifies the pictures using some filters graphics application
- Bob travels with his parents and meanwhile shows the pictures



Replication – Consistency

An example of inconsistency

- Alice accesses DB1, since this RM is closer and replies faster
- Alice updates X to X’



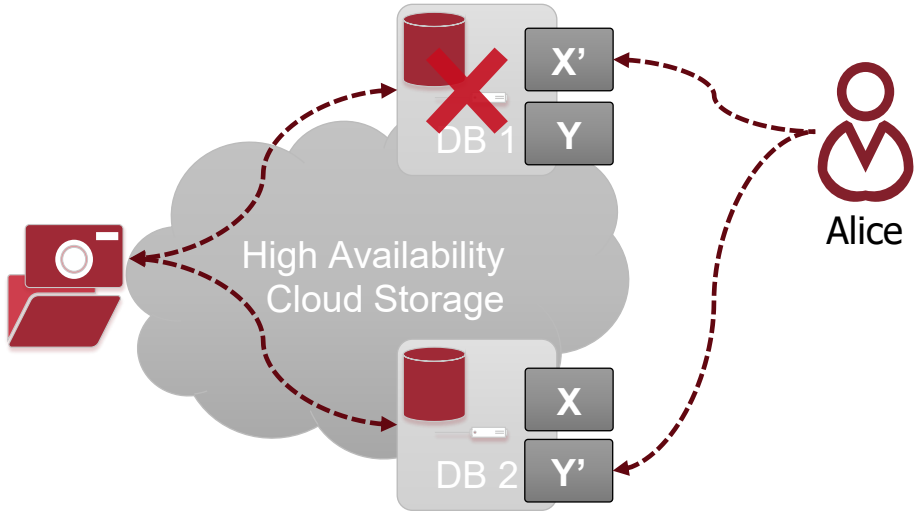
Replication – Example of inconsistency

Alice accesses DB1, since this RM is closer and replies faster

Alice updates X to X'

Alice tries to update Y to Y', she notices that DB1 has failed and thus connects to DB2

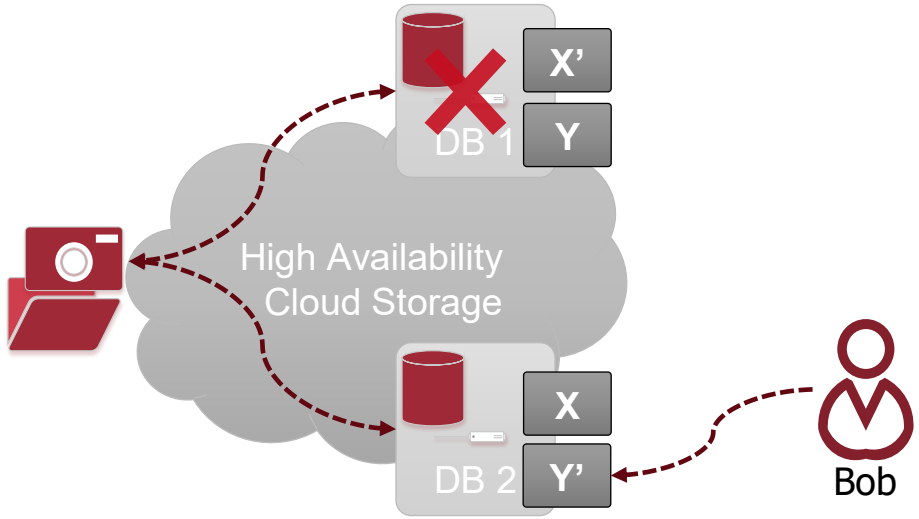
Note that DB1 has failed before propagating the update of X to DB2



Replication – Example of inconsistency

Since Bob is traveling, he accesses DB2 which is closer

Bob first shows Y, which means he presents the updated version Y’ to his parents

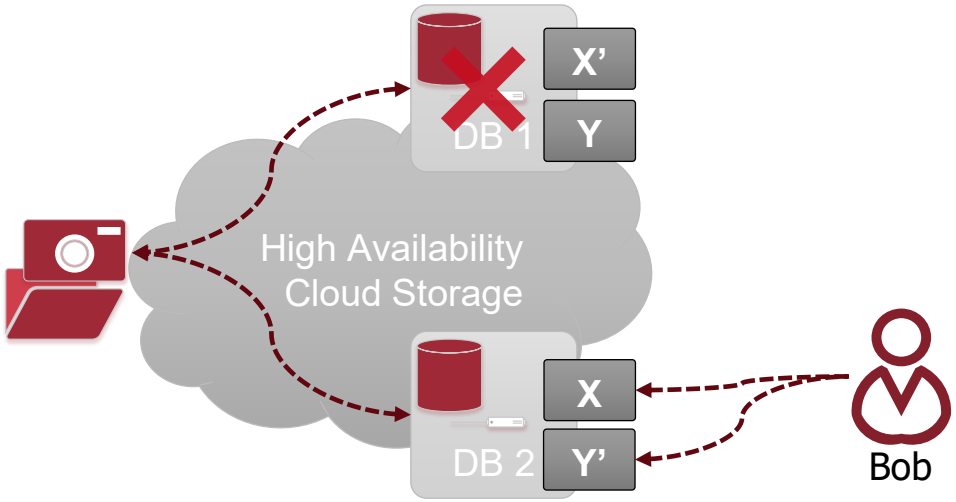


Replication – Example of inconsistency

Afterwards he shows X, which means he presents the outdated (bad quality) version

This situation is inconsistent, because

- If one update occurred after another, then the first update should be observed before the second has been observed
- Bob observed Y' but missed X'



Replication – Consistency

Keeping a system consistent = trade-off between scalability and consistency

Consistency model = contract between processes/clients and distributed data store

Client expects that the result of a read operation on a data item reflects the effects of the last write on that data item

Two Views On Consistency

Data-centric consistency models

- Talk about consistency from a global perspective
- Provides guarantees how a sequence of read/write operations are perceived by multiple clients

Client-centric consistency models

- Talk about consistency from a client‘s perspective
- Provides guarantees how a single client perceives the state of a replicated data item

Data-Centric Consistency Models

Strong consistency models: Operations on shared data are synchronized

- Strict consistency (related to time)
- Sequential consistency (what we are used to)
- Causal consistency (maintains only causal relations)
- …

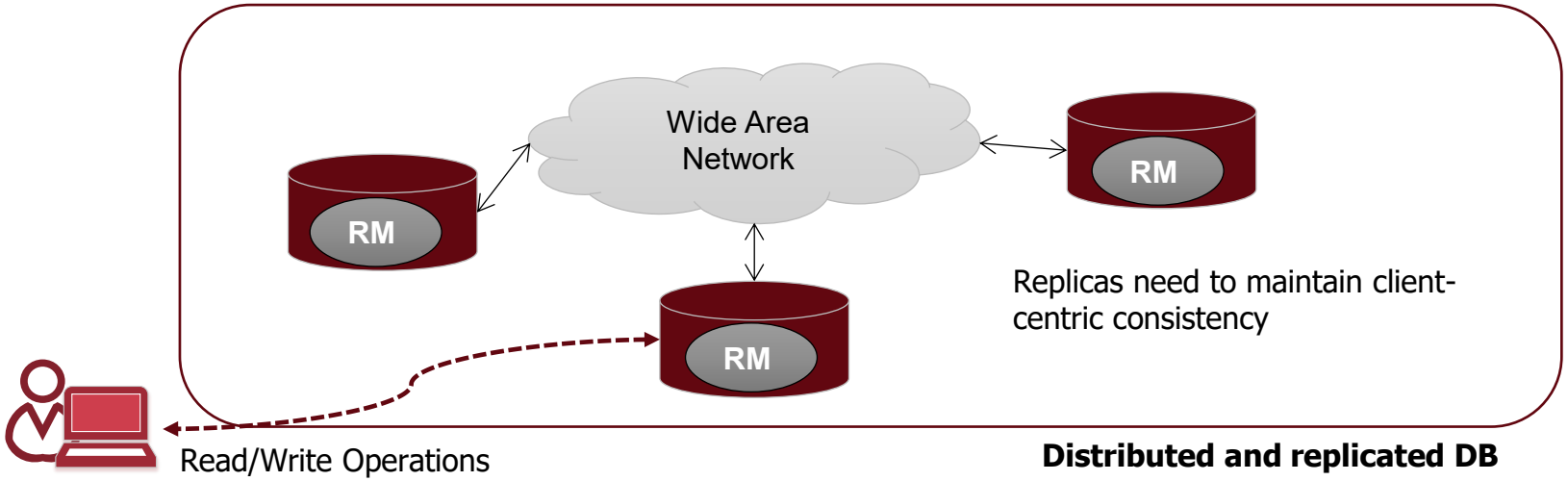
Weak consistency models: Synchronization only when data is locked/unlocked

- General weak consistency
- Release consistency
- Entry consistency
- …

Client Centric Consistency

Useful for large-scale, distributed DBs that tolerate high degree of inconsistency (e.g. DNS or web-caches)

Consistency guarantees from the point of view of a single client



Consistency Models

Real world cloud services provide foundation and resources to build Internet-scale, Cloud-based applications

- High requirements on security, scalability, availability, performance
- Meet requirements while serving millions of customers
- Service are massive distributed systems → failures will happen!
- Replication techniques are used to guarantee performance and availability
- Consequences of using replication → tradeoff between availability and consistency

Consistency Models

Ideal world consistency model: all observers see the results of an update

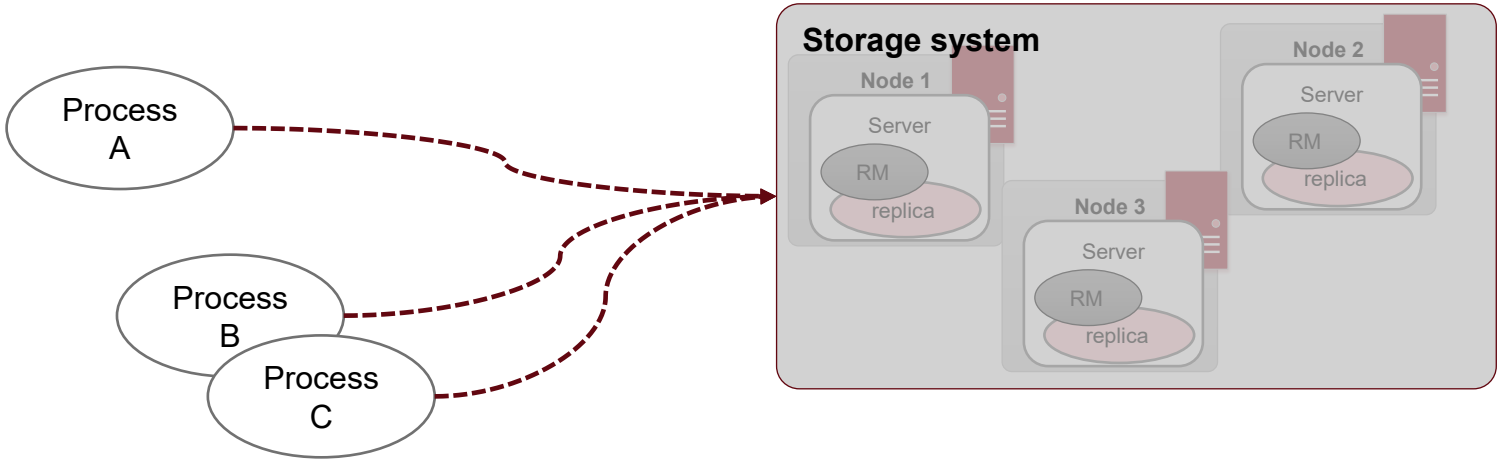
- Difficult to achieve in large-scale systems → is it better to fail the whole system than to break distribution (replication) transparency?
- How to make trade-offs? → CAP Theorem
- Network partitions occur in large scale applications spanning data centers → cannot reach consistency and availability → relax consistency or drop availability
- Both cases require the client developer to be aware of it
  - Relax consistency: accept all writes, but read may not return result of a recently completed write
  - Drop availability: handle situations where the system cannot accept a write

Client-centric consistency: Scenario

Storage system (large scale and highly distributed)

Process A: writes to and reads from the storage system

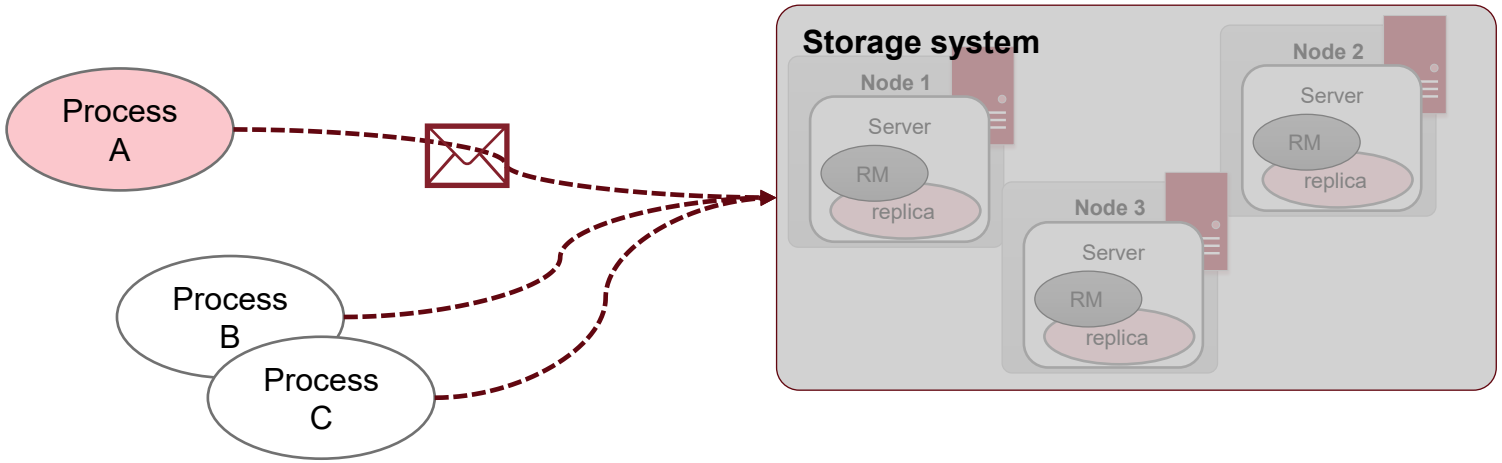
Process B and C: write and read form the storage system independent of A



Client-centric consistency

Consistency from the client/developer POV

- Client-side consistency is about how and when processes A, B and C see updates made to data objects in the storage system
- Consider process A making an update:



Strong and weak consistency

Strong consistency: any subsequent access from A, B or C returns updated value

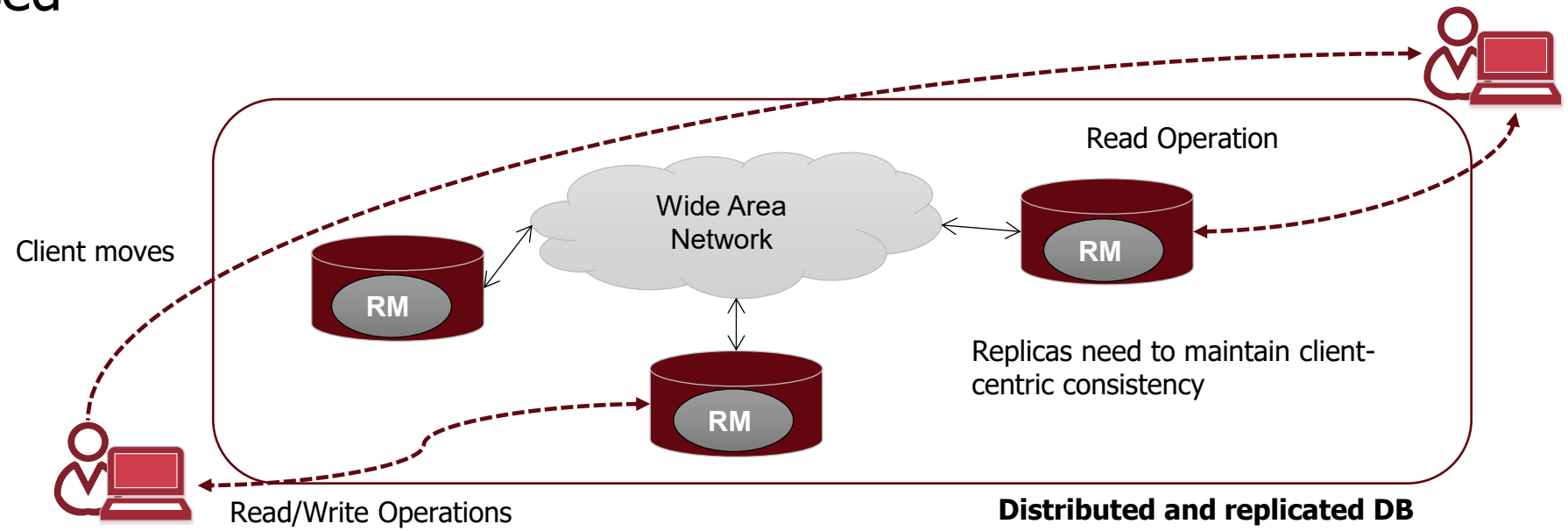
Weak consistency: storage system does not guarantee that a subsequent access will return the updated value → inconsistency window exists (period between update and guarantee to access updated value)

- Eventual consistency: If no failures occur the system will eventually become consistent, max size of inconsistency window can be determined
- Causal consistency: consider process A has communicated update to B, but not to C
  - Access by B must return updated value
  - C has no causal relation → update does not necessarily have to be seen

Client-centric consistency: Read-your-writes consistency

Read-your-writes: if process A updated value, it will never see an older version

Problem: applications of mobile users are usually unaware which RM is accessed



Client-centric consistency

Important variations (continued):

- Session consistency: practical version or read-your-writes as process accesses storage system in session, guarantees hold as long as session is not terminated
- Monotonic read consistency: if a process has seen a value, a subsequent read will never return an older version
- Monotonic write consistency: writes by a process are serialized (applied in the same order as issued by the client)

Quorum consistency

Consistency from the server point of view:

- N: number of nodes the storage system consists of
- W: number of replicas that need to acknowledge updates
- R: number of replicas that are involved in a read operation
- If W+R > N, read and write set overlap and strong consistency can be guaranteed
  - Smaller partition is not available
  - If more than W nodes fail, full system is unavailable
  - Unavailable systems will have to fail write operations!
- If a system uses delayed (asynchronous) updates and R+W <= N (e.g. N=2, R=1, W=1), consistency cannot be guaranteed

Notes on consistency

Consistency from the server point of view:

- How to configure N, W, R depends on applications requirements
- R=1 and W=N optimized for reads
- W=1 and R=N for writes
- Weak/eventual consistency is given if W+R <= N (read and write sets do not overlap)
- System that focus on fault tolerance often use N=3, W=2, R=2
- Systems that serve very high loads often use N>10… with R=1
- If weak consistency is desired, usually R=1

Notes on consistency

Implementation of consistency is often related to the “stickiness” of a client

Using sessions, a client is bound to a server → read-your-writes and monotonic reads can be achieved easily

- Load-balancing and fault-tolerance more complicated

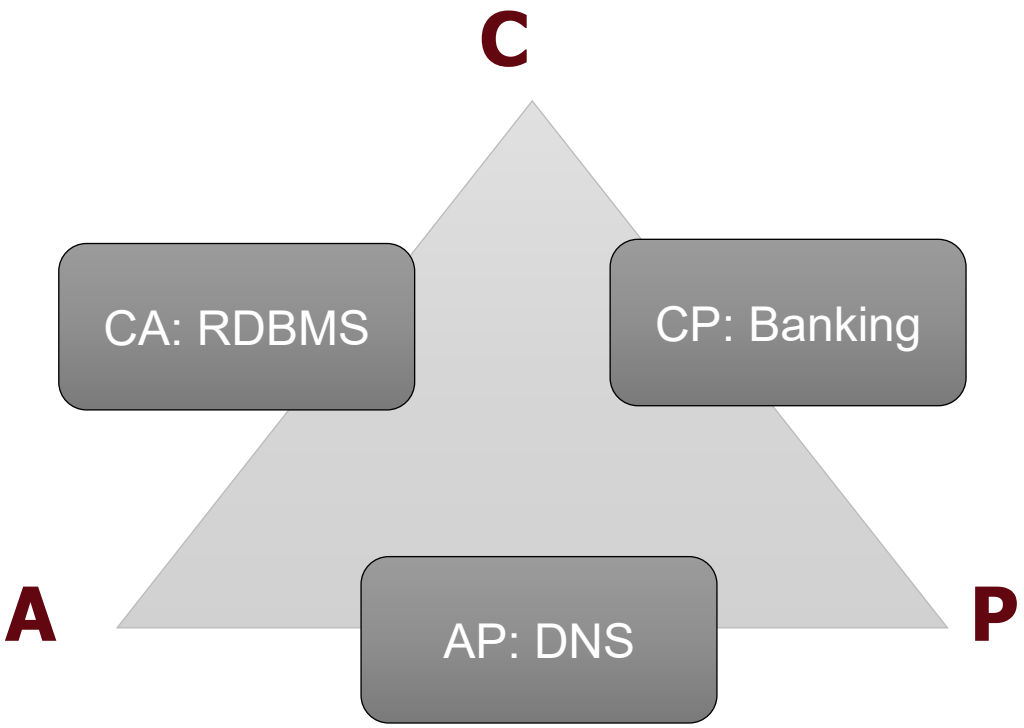
Clients can implement read-your-writes and monotonic reads by adding versions on writes (reads with lower versions are discarded) → remember end-to-end arguments in system design, it is important where to place logic

CAP theorem

CAP theorem (Brewer‘s theorem) defines that a DS can only satisfy two of the following three properties at once:

- Consistency: All replicas of a data item are consistent. A system must ensure that after updating a data item, all of its replicas are updated as well. A read operation will return the value of the most recent write operation
- Availability: The systems ability to respond to requests (within an acceptable amount of time)
- Partition Tolerance: The systems ability to keep operating if nodes or communication channels fail

CAP theorem



CAP theorem

AP: Domain Name System

- The DNS is one of the core Internet services
- It is designed to be very resilient against failures of single nodes and provides high availability (ever recognized a crash of the DNS system?)
- However, its hierarchic design and extensive use of replicas and local caches lets it provide a very weak level of consistency (eventual consistency)
- It can take days until modified DNS entries are propagated to all replicas and caches

CAP theorem

CP: Banking

- Banks avoid inconsistencies at any cost.
- To ensure consistency, the availability is reduced by introducing up to 3 days delay of pending transactions.
- Technically, ensuring consistent results from all involved banking servers does not require 3 days of unavailability.
- Banks use the blocked money in the meantime.

CAP theorem

CA: Relational Database Management System (RDBMS, e.g. MySQL, Oracle, ..)

- RDBMS is consistent and designed to be available because it is a foundation of a lot of business-critical applications
- What if we would try to add partition tolerance? → the RDBMS waits for a failed node to come back to ensure consistency. Because maybe just the communication channel has failed!
  - No requests can be served in the meantime
  - We lose availability!
  - Mitigation for missing P: RDBMS often built using high availability hardware

Can we really choose so sacrifice partition tolerance?

Summary

Availability is often (especially in Cloud based environments) more desirable than strong consistency → consistency must be relaxed since partitions cannot be avoided

Classic quorum approaches tolerate the unavailability of small partitions

→ eventual consistency allows for inconsistent data objects

→ merge operation (partition healing) must be introduced → application-level logic

Questions ?